What an amplitude-estimation error bar actually controls

Pradyot Bathuri

23 October 2026

Framing and reference values

A certified estimate of a badly encoded amplitude is not a certified price or CVaR. This project connects my work on regime-conditional factor models with quantum amplitude estimation by asking a numerical-analysis question: which part of the final error does the estimator's a priori bar control? The examples separate the continuous or empirical target, its discrete encoding, and estimation of the encoded expectation. The interesting results are the size of each error contribution, a quadrature bias that survives refinement, and a noise bias that grows with circuit size. The calculations check a known algorithm; they do not constitute an interval-arithmetic proof or a new pricing method.

The call has $S_0 = 100$, $K = 105$, $r = 0.05$, $\sigma = 0.2$ and $T = 1$ year. With Φ and ϕ the standard-normal distribution and density, its Black-Scholes value is

$$C = S_0\Phi(d_1) - Ke^{-rT}\Phi(d_2) = 8.0213522351, \qquad d_{1,2} = \frac{\log(S_0/K) + (r \pm \sigma^2/2)T}{\sigma\sqrt{T}}.$$

For a normal loss $L \sim N(\mu, \sigma_L^2)$, the second reference is

$$q_\alpha = \mu + \sigma_L\Phi^{-1}(\alpha), \qquad \mathrm{CVaR}_\alpha(L) = \mu + \frac{\sigma_L\phi(\Phi^{-1}(\alpha))}{1-\alpha}.$$

At $\mu = 0$, $\sigma_L = 1$ and $\alpha = 0.95$, VaR is 1.6448536270 and CVaR is 2.0627128075, in standard-normal loss units. These formulas and parameters are implemented in `problems.py` and checked in `test_audit.py`; the fresh `run.py` output is saved in `verification/run_report.txt`.

For masses p_i and encoded values $0 \le f_i \le 1$, the marked probability is $a = \sum_i p_i f_i$. Canonical amplitude estimation returns $\hat{a} = \sin^2(\pi k/t)$, where $t = 2^m$ is the number of phase bins. The Brassard-Hoyer-Mosca-Tapp inequality used in `estimation.py` is

$$\Pr\{|\hat{a} - a| \le B_t(a)\} \ge \frac{8}{\pi^2} = 0.810569\ldots, \quad B_t(a) = \frac{2\pi\sqrt{a(1-a)}}{t} + \frac{\pi^2}{t^2}.$$

This is an ideal-oracle, probabilistic a priori bound. Here the exact encoded a is available for auditing. If it were unknown, the conservative bound $B_t(a) \le \pi/t + \pi^2/t^2$ follows from $a(1-a) \le 1/4$; the plotted, amplitude-dependent bars are not confidence intervals inferred from observed shots. Multiplication by the payoff scale converts the amplitude bar to the units of the reported quantity. It does not bound quadrature, quantile, sampling or hardware error.

At fixed success probability and fixed encoding, the ideal query improvement is quadratic: order $1/\epsilon$ oracle uses versus the usual Monte Carlo order $1/\epsilon^2$, not exponential. The controlled powers here use Q a total of $t-1$ times. Loading data and compiling those powers also cost work. Statevector simulation supplies no wall-clock quantum speedup. Expectation and CVaR estimation are established applications: Montanaro (2015), Woerner and Egger (2019), and Tanaka et al. (2021) provide the relevant background; references are listed on the last page.

Quadrature: refinement changes only one part of the error

Let $C_h = sa_h$ be the encoded grid price and $\widehat{C}_h = s\hat{a}_h$ its amplitude estimate, with s the discounted payoff maximum. Then

$$\widehat{C}_h - C = (\widehat{C}_h - C_h) + (C_h - C).$$

The first term is controlled by $sB_t(a_h)$ with the stated probability. The second is the deterministic encoding error. A price bound requires both terms. In this benchmark the closed form makes the second term measurable; it has not been enclosed by interval arithmetic.

The uniform loader uses equally spaced **spot** nodes across a log-price window corresponding to standard-normal shocks in $[-4, 4]$. Masses are proportional to the lognormal density and normalized to sum to one. Its limiting integral is therefore the windowed, renormalized expectation, $C_W = 8.0177885991$. Splitting once more gives

$$C_h - C = (C_h - C_W) + (C_W - C), \qquad C_W - C = -0.0035636360.$$

Additional nodes can reduce $C_h - C_W$ but cannot remove the window bias. "Floor" here means the asymptotic absolute bias, not a lower bound on every finite-grid error. At 96 nodes the spot-grid error is only 0.00061609 dollars: quadrature error temporarily cancels some window bias. At 256 nodes its price is 8.0178887418, already close to the biased limit.

Gauss-Hermite integrates over the full Gaussian domain. Its polynomial exactness does not make the call payoff smooth at the strike. The saved sweep shows both a worse 16-node result and non-monotone error with refinement. All prices and signed errors in the table are in dollars:

Nodes	Gauss-Hermite price	Signed error	Uniform-grid price	Signed error
16	8.19192793	+0.17057570	8.00851991	-0.01283233
64	8.03695667	+0.01560444	8.01769175	-0.00366048
96	8.02005137	-0.00130087	8.02073615	-0.00061609
128	8.01162193	-0.00973030	8.01852537	-0.00282686
256	7.99900672	-0.02234551	8.01788874	-0.00346349
512	8.01974650	-0.00160574	8.01775543	-0.00359680

Source: `data/scale_sweep.json`, `classical`; errors are against Black-Scholes. This comparison concerns this payoff and these nodes, not a general ranking of quadrature rules. A small error at one node count is insufficient evidence of convergence, particularly when two signed contributions can cancel.

The upper panels of `figures/scaling_methods.png` show these absolute errors and a separate Monte Carlo sweep. Monte Carlo uses 24 seeds at each path count. The dashed standard-error reference uses a payoff standard deviation estimated from a separate simulation; it is not a rigorous finite-sample confidence bound. Node count, path count and phase-bin count appear on separate axes because they are different computational budgets. The figure does not establish equal end-to-end costs.

Figure 1 caption: source `data/scale_sweep.json`, regenerated 23 October 2026; its noisy panel uses the 15 April 2026 FakeKingston calibration in Qiskit Aer, without mitigation, not live hardware.

Amplitude estimation against its own bar

For $a = \sin^2(0.7) = 0.4150164285$, the statevector calculation gives the entire outcome distribution. "Median" below means its probability-weighted median, computed classically, not a single measurement or a median from an experimental repetition protocol.

Phase bins t	Absolute median error	Amplitude bar	Probability inside bar
16	0.08498357	0.23204574	0.828128
64	0.01256159	0.05078272	0.894985
128	0.01161833	0.02478897	0.811465
256	0.00049737	0.01224389	0.996277
1024	0.00049737	0.00303273	0.949507

The success probability can decrease while the bound narrows. The median can also remain on the same phase-bin value across refinements. These features are visible in the exact distribution; they are not simulation failures. The smallest probability in this table is still above $8/\pi^2$.

The 16-node call has $a_h = 0.0677149409$ and scale $s = 118.268137$ dollars. The correct reference for its amplitude bar is the grid price 8.0085199051, not Black-Scholes.

t	Median price	Error vs grid	Price bar	Probability inside
16	4.501313	-3.507207	16.228894	0.921119
64	6.982476	-1.026044	3.202297	0.843049
256	7.682285	-0.326235	0.747141	0.814712
1024	8.043838	+0.035319	0.183446	0.933545

Source for both tables: `data/scale_sweep.json`, `amplitude_estimation`. Call prices, signed errors and bars are in dollars. At $t = 16$ the bar is wider than the price itself. At $t = 1024$, the median's Black-Scholes error is +0.022486 dollars, combining its positive grid-estimation error with the negative grid bias. The success probabilities concern a random phase-estimation outcome; the tabulated medians are descriptive summaries.

The seeded one-shot outcomes for this known amplitude and the call are inside their bars, but a miss is retained in the Gaussian CVaR example. With 20,000 losses, seed 11, and 32 bins, the exact histogram excess assembly is 2.0603234871. At $t = 16$, its median estimate is 3.2189159024, its propagated bar is 3.245422, and the seed-0 single shot is **outside** that bar. Its exact success probability is 0.858922, consistent with the theorem. At $t = 1024$, the median is 2.0606839698 with bar 0.02604439. These values and the miss are reproduced by `run.py` and protected by the existing audit.

The left panel of `figures/scaling_details.png` also retains the known-amplitude single-shot errors. There is no filtering of failed draws or replacement of a single-shot guarantee by a claim that every draw succeeds.

Figure 2 caption: source `data/scale_sweep.json`, regenerated 23 October 2026; the no-CZ panel is unmitigated Aer simulation of the 15 April 2026 FakeKingston calibration, not live hardware.

Simulated noise: a separate error contribution

All noisy results use **FakeKingston, the ibm_kingston calibration snapshot dated 15 April 2026, in Qiskit Aer**. They are not live quantum-device measurements. No error mitigation is applied. `MEASUREMENTS.md` documents the snapshot and the earlier runs; the scaling table below uses the separate sweep in `data/scale_sweep.json`, averaging eight seeds with 4,096 shots each. These circuits prepare and measure the call loader without amplitude amplification.

Spot nodes	CZ gates	Grid price	Ideal Aer mean	Noisy Aer mean	Noisy minus grid
4	7	4.017360	3.981011	5.688189	+1.670829
8	23	8.037867	8.034206	10.585951	+2.548085
16	62	8.008520	7.860962	12.816472	+4.807952
32	132	7.983626	7.850134	17.761154	+9.777529

Prices and shifts are in dollars. The last column separates the observed mean shift from the grid bias. That shift grows with CZ count in this sweep. Node count, circuit depth and the compiled circuit also change, so this is not an experiment isolating CZ errors as the only cause. More shots would reduce sampling fluctuations without correcting the noise model's systematic displacement.

A simple diagnostic in `MEASUREMENTS.md` mixes the ideal marked probability with $1/2$. If p is a representative CZ error and g is the CZ count, set

$$w = (1-p)^g, \qquad a_{\mathrm{mix}} = wa + (1-w)/2.$$

For $p = 0.0016$ and $g = 62$, $w \simeq 0.9055$, and $a = 0.0677$ gives $a_{\mathrm{mix}} \simeq 0.109$. The scaling sweep's measured noisy frequency is 0.108368. This survival-and-mixing calculation is a rough consistency check, not a fitted channel model, a proof of mechanism, or an error bound. It omits details of relaxation, readout and the individual gates. It is not used to adjust any price.

The one-qubit sequence is a useful control because it has no CZ gates. Its ideal marked probability is $\sin^2((2m+1)\theta)$ with $\theta = 0.7$. In the scaling sweep, at $m = 4$ the ideal probability is 0.00028271 and the mean noisy frequency is 0.01234436. Thus even this control has a visible noise floor near a zero of the ideal oscillation. Across the saved sequence, the oscillation largely survives. The sequence averages 16 seeds with 4,096 shots each and reaches $m = 16$; it does not test multiqubit call-state preparation.

The earlier canonical phase-estimation experiment in `MEASUREMENTS.md` is also cautionary: at $t = 4$ and $t = 8$, ideal and noisy median estimates both equal $1/2$. Their error against $\sin^2(0.7)$ is about 0.08498, while the ideal bars are about 1.391 and 0.541. Passing such wide bars provides little diagnostic information, and the ideal theorem does not certify the noisy channel. Agreement of a coarse median can conceal changes in the outcome distribution.

Regime-conditional CVaR: distinguish the three targets

The empirical application uses losses equal to minus monthly returns. At $\alpha = 0.95$, the code retains three distinct quantities: the sample tail mean S, the legacy comparator-selected histogram mean H, and the histogram excess assembly D. With the sample quantile q, histogram centers c_i, and masses p_i, the amplitude routine estimates

$$D = q + \frac{\sum_i p_i (c_i - q)_+}{1 - \alpha}, \qquad \widehat{D} = q + \frac{E_{\max}}{1 - \alpha} \hat{a}, \quad E_{\max} = \max_i (c_i - q)_+.$$

Its bar is $E_{\max} B_t(a)/(1-\alpha)$. It controls $\widehat{D} - D$, conditional on the fixed sample quantile and encoding. The useful empirical decomposition is

$$\widehat{D} - S = (\widehat{D} - D) + (D - S).$$

The second term combines binning with finite-sample threshold and tail-averaging conventions. `sample_cvar` averages observations at or above `numpy.quantile`'s threshold; that need not equal the empirical excess formula with divisor $1 - \alpha$. Nor is the sample quantile necessarily the histogram's own quantile. Calling $D - S$ pure quantum error, or calling D the exact histogram CVaR, would misstate the target. Population sampling error and regime-label uncertainty remain outside the displayed bar.

The legacy H stays deliberately unchanged. Its comparator uses width $(\text{hi} - \text{lo})/(2^n - 1)$ while the histogram width is $(\text{hi} - \text{lo})/2^n$. On the Gaussian sample, the selected tail has mass 0.06835 rather than 0.05; $H = 1.9303992869$, versus $S = 2.0606125246$. It does not use the amplitude-estimation output. Estimating its selected-tail probability accurately cannot repair the returned conditional mean. The middle panel of Figure 2 preserves this baseline.

The saved CRSP/Compustat panel contains 3,090,082 firm-months and 25,048 identifiers over 618 months, July 1973 through December 2024. `panel_cvar.py` forms equal-weighted size and book-to-market quintile portfolios using NYSE breakpoints and requires at least ten names per portfolio month. Each reported quintile has 221 high-volatility and 385 low-volatility months. Delisting returns are absent from the local pull. These are descriptive conditional samples, not out-of-sample risk guarantees.

Selected portfolio results from `data/panel_cvar.json`, in **percent monthly loss**; the bar is in **percentage points**, at $t = 1024$:

Portfolio / regime	Sample S	Legacy H	Excess D	AE median $\widehat{D}$	AE bar
Size Q1 / high	17.00793	16.40728	17.72510	17.68646	0.27221
Size Q1 / low	9.34103	8.44610	9.40425	9.44051	0.12100
Book/market Q5 / high	17.10908	14.52532	17.57854	17.48918	0.29696

All 20 portfolio medians at this t are inside their bars relative to D. Only seven of the sample values S lie in those same AE-centered intervals. The distinction is visible without claiming any population coverage. `figures/panel_cvar.png` shows sample S for all quintiles, with no inferred confidence intervals.

Figure 3 caption: source `data/panel_cvar.json`, July 1973-December 2024 panel; regenerated 23 October 2026; equal-weighted portfolio losses, split by the saved volatility labels.

Market series, reproduction and the remaining question

The market calculation in `market_check.py` applies the same decomposition to loss = −mktrf, the stored monthly market excess return. Its 634 labeled months comprise 223 high-volatility and 411 low-volatility months. These are not the same eligible windows as the CRSP quintile portfolios. With no closed-form population reference, the bar still concerns only the fixed histogram excess assembly.

From `data/market_regimes.json`, in **percent monthly excess-return loss**; bars are **percentage points**, at $t = 1024$:

Regime	Months	Sample S	Legacy H	Excess D	AE median	AE bar
All labeled	634	10.36406	10.04906	10.30354	10.30477	0.17646
High volatility	223	13.14417	13.16031	13.40383	13.37826	0.18181
Low volatility	411	7.56905	7.40225	7.62960	7.66911	0.09236

For example, high-volatility $D - S$ is about 0.25966 percentage points, while the median's error against D is about −0.02557 points. The amplitude bar does not cover the larger target mismatch. Both the high- and low-volatility sample values fall outside the corresponding AE-centered bars, even though their AE medians meet the bound against D.

The panel also contains firm-level results for histories with at least 60 months in a regime. The amplitude summary comprises 17,468 **firm-regime pairs**, not distinct firms, at $t = 256$. Every recorded median lies inside its bar; the median propagated bar is 0.01072510 in decimal monthly-loss units, or 1.07251 percentage points. This is an audit of the exact distributions' medians, not an empirical single-shot coverage rate.

IU Big Red 200 job **8688102**, dated 22 October 2026 in `BIGRED_RUN.md`, reran `panel_cvar.py --firm-ae` on the AMD EPYC system. The saved `.out` records the cluster output path and matching reported panel results; the `.err` is empty. A direct comparison of the two saved JSONs finds agreement at reported precision, with a maximum absolute numeric difference of 1.33×10^{-15}. Thus the note's "last digit" description should be understood at reporting precision, not as literal JSON identity. The cluster JSON still says `machine: local Mac`; job provenance comes from the run record, not that field. Big Red reproduced the calculation on a second architecture. It did not make the noise more real or demonstrate faster mathematics.

Could an Evans-function or winding-number viewpoint offer a useful way to organize an estimator's error bar: first identify the object being certified, then determine whether numerical perturbations preserve the conclusion, in the spirit of a Rouche argument? I do not know whether that viewpoint yields anything beyond the decomposition here. This is a question for discussion, not a claimed connection or an implemented Evans function.

Verification and sources. On 23 October 2026, the existing numerical audit passed all 13 tests and the Qiskit audit passed all five. A fresh `scale_sweep.py` execution, redirected to a separate verification file, reproduced the saved sweep JSON exactly. `plot_scale.py` regenerated both scaling figures from the preserved JSON; `plot_panel.py` generated the sole additional figure. Protected data, tests and existing logs were checked by SHA-256. The local numerical sources are `NOTE.md`, `MEASUREMENTS.md`, the three application/sweep JSONs, `problems.py`, `estimation.py`, `run.py`, and the Big Red record. No underlying panel data were regenerated.

Established work. Montanaro, *Quantum speedup of Monte Carlo methods* (2015); Woerner and Egger, *Quantum risk analysis* (2019); Tanaka et al., *Amplitude estimation via maximum likelihood on noisy quantum computer* (2021). These references establish prior work, not novelty for the calculations reported here.